

Trabajo Práctico 8 — El jefe aprende a pensar (máquina de estados)

Diplomatura de Videojuegos · Clase 8 · Proyecto final Objetivo: darle **cerebro al slime elite** (el jefe) del TP7 con una **máquina de estados**: te va a **acechar** de lejos, **perseguirte** cuando te acercás, **atacarte** cuando te alcanza, y **huir** cuando le queda poca vida. El slime básico **queda como está**: es la horda. Y al final, **exportás** el juego: es tu proyecto final.

Qué vas a lograr

- El **jefe** con **cuatro estados** — `ACECHAR`, `PERSEGUIR`, `ATACAR`, `HUIR` — y las transiciones entre ellos, escritas con `enum` + `match` como en la clase.
- Una **etiqueta sobre el jefe** que muestra en qué estado está: vas a ver la máquina de estados funcionando.
- El jefe **ya no es kamikaze**: se frena y te pega **cada segundo** mientras estés cerca.
- El slime básico **intacto**: sigue siendo la horda simple del TP7.
- El juego **exportado** a `.exe`, listo para compartir.

 **Tiempo estimado:** 75–100 min. Se toca **un solo script**: `enemigo_elite.gd`. El slime básico, el jugador, las balas y el spawner quedan como estaban.

 **Viene de la Clase 8:** comportamientos básicos, el problema del spaghetti de `if`, máquinas de estado, `enum` y `match`, y el patrón **hacer + decidir**. Y de todo el curso: herencia y `super()` (TP5), `Timer`, grupos y `distance_to` (TP7).

Punto de partida

Este TP continúa `tp7-sobrevivir`. Abrilo (o copió la carpeta como `tp8-final`).

Confirmá que el **punto de control 5 del TP7** sigue andando: los slimes te persiguen, las balas salen solas, y cada 8 aparece un elite con su barra.

🗨️ **Recordá cómo quedó el elite en el TP7.** `enemigo_elite.gd` hereda de `enemigo.gd` (`extends "res://enemigo.gd"`): en `_ready()` llama a `super()` y después cambia `vida` , `velocidad` , `dano` y su `$BarraVida` ; y en `recibir_dano()` llama a `super(cantidad)` y actualiza la barra. Todo lo demás — perseguir en `_process` , `morir()` , y el `_on_body_entered()` kamikaze— lo **hereda tal cual** del básico. Hoy le vamos a dar **su propio** `_process` .

🗨️ Decisión de diseño: ¿por qué solo el jefe?

Si le diéramos la máquina de estados a **todos** los slimes, ¿cuál sería su estado “tranquilo”? La opción clásica es **patrullar** (caminar al azar), pero en un *survivors* sería un error: los slimes se irían **fuera de la pantalla**, el jugador podría **evitarlos para siempre**, y el juego se llenaría de enemigos que nadie ve y que igual gastan procesador.

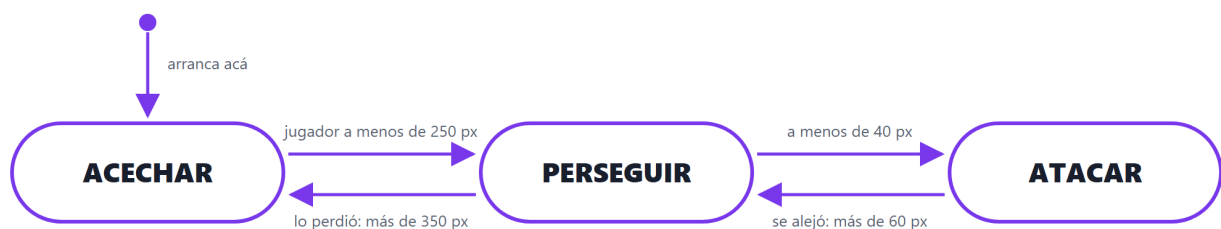
Por eso:

- La **horda** (slime básico) sigue **simple y kamikaze**: siempre viene por vos. Es previsible a propósito.
- El **jefe** es el que **piensa**. Y su estado tranquilo **no es patrullar**: es **acechar** — venir hacia vos, pero **despacio**. Nunca se pierde, nunca se va de pantalla.

Fijate que la decisión no es técnica, es de **diseño de juego**: la IA tiene que hacer el juego más divertido, no más “inteligente” (Clase 8).

🧩 El plan: primero el diagrama

En la clase dijimos: **si no lo podés dibujar, no lo podés programar**. Este es el diagrama que vamos a implementar en el jefe, con los mismos umbrales de la clase:



La máquina del slime elite (jefe) · cada óvalo es un estado · cada flecha, una transición · solo UN estado activo a la vez

Y lo vamos a construir **de a un estado por vez**, probando cada uno antes de seguir:

PARTE	ESTADOS	QUÉ SE VE
1	ACECHAR ⇌ PERSEGUIR	El jefe viene despacio; al acercarte, acelera
2	+ ATACAR	Se frena al alcanzarte y te pega cada segundo
3	+ HUIR	Escapa cuando le queda 1 de vida
4	—	Ajustar, exportar y entregar



Parte 0 — Dibujar antes de programar

Antes de escribir una línea, respondé estas tres preguntas **mirando el diagrama** (en papel o mentalmente):

1. El jefe está **acechando** y el jugador pasa a **200 px**. ¿En qué estado queda?
2. El jefe está **persiguiendo** y el jugador está a **300 px**. ¿Cambia de estado?
3. El jefe está **atacando** y el jugador se aleja a **50 px**. ¿Cambia? ¿Y si se aleja a **70 px**?

▼ Respuestas (abrí después de pensarlas)

1. **PERSEGUIR** : 200 es menos de 250, así que cruza la flecha de "lo vio".
2. **No**. 300 no es menos de 40 (no ataca) ni más de 350 (no lo perdió). **Se queda persiguiendo**. Las flechas solo se cruzan cuando se cumple su condición.
3. A **50 px sigue atacando** (50 no es más de 60). A **70 px vuelve a PERSEGUIR** .



¿Por qué entra a atacar a 40 y sale a 60, y no a 40 en los dos? Si el umbral fuera el mismo, un jugador parado justo en el borde haría que el jefe **parpadee** entre atacar y perseguir cada frame. Dejar un margen entre "entrar" y "salir" se llama **histéresis**, y es un truco que vas a usar en toda máquina de estados.

✅ **Punto de control 0:** contestaste las tres, entendés que cada flecha tiene **su** condición, y por qué los umbrales de entrada y salida son distintos.



Parte 1 — Dos estados: ACECHAR y PERSEGUIR

Concepto: el jefe deja de usar el `_process` del slime básico y recibe **el suyo**, con la máquina de estados. Arrancamos con **dos** estados, y una **etiqueta** encima que muestra cuál está activo.

Reemplazá `enemigo_elite.gd` por esta versión completa:

```

extends "res://enemigo.gd"

enum Estado { ACECHAR, PERSEGUIR }

var estado := Estado.ACECHAR
var jugador: Node2D = null

func _ready() -> void:
    super() # grupo, señal y animación del slime básico
    vida = 5
    velocidad = 35.0
    dano = 25
    $BarraVida.max_value = vida
    $BarraVida.value = vida

    jugador = get_tree().get_first_node_in_group("jugador")

    # Etiqueta que muestra el estado, creada por código
    var etiqueta := Label.new()
    etiqueta.name = "LabelEstado"
    etiqueta.position = Vector2(-36, -66)
    add_child(etiqueta)

func _process(delta: float) -> void: # SOBREScribe el _process del básico
    if jugador == null:
        return
    var d := distancia_al_jugador()
    match estado:
        Estado.ACECHAR:
            acechar(delta) # hacer
            if d < 250: estado = Estado.PERSEGUIR # decidir
        Estado.PERSEGUIR:
            perseguir(delta)
            if d > 350: estado = Estado.ACECHAR
    $LabelEstado.text = Estado.keys()[estado] # mostrar el estado encima

func _on_body_entered(body: Node) -> void: # ANULA el kamikaze del básico
    pass

# ---- comportamientos: cada estado, una función ----
func acechar(delta: float) -> void: # hacia el jugador, a MITAD de velocidad
    var dir := (jugador.position - position).normalized()
    position += dir * velocidad * 0.5 * delta
    $AnimatedSprite2D.flip_h = dir.x < 0

```

```
func perseguir(delta: float) -> void:           # hacia el jugador, a toda velocidad
    var dir := (jugador.position - position).normalized()
    position += dir * velocidad * delta
    $AnimatedSprite2D.flip_h = dir.x < 0

func distancia_al_jugador() -> float:
    return position.distance_to(jugador.position)

# ---- vida: igual que en el TP7 ----
func recibir_dano(cantidad: int) -> void:
    super(cantidad)
    $BarraVida.value = vida
```

🗨 Qué está pasando acá (tres ideas de herencia):

- **Sobreescribir `_process`** . Al definirlo en el hijo, el del padre **deja de correr** para el jefe (a propósito no llamamos `super()` ahí). Los slimes básicos siguen usando el del padre: por eso no cambian.
- **Anular `_on_body_entered`** . El padre lo conecta a la señal en su `_ready()` (que sí llamamos con `super()`), pero como el jefe lo **redefine vacío** (`pass`), al tocarte no pasa nada. El ataque va a llegar por un **estado**, en la Parte 2.
- `Estado.keys()[estado]` devuelve el nombre del estado como texto (`"ACECHAR"`). Eso muestra la etiqueta. Es *la* herramienta para depurar una máquina de estados: si algo anda raro, mirás la etiqueta y sabés exactamente en qué estado está.

Y el patrón de siempre: en cada estado, primero **hacer** (llamar a la función), después **decidir** (¿cambio?).

Probarlo rápido: esperar 8 slimes cada vez es lento. Para testear, en `nivel.tscn` seleccioná `Enemigos` e **instanciá** un `enemigo_elite.tscn` a mano (ícono de cadena), ubicado lejos del jugador. Cuando termines el TP, borrá esa instancia.

Apretá **F6**. El jefe viene hacia vos **despacio** con `ACECHAR` encima. Acercate a menos de 250 px: la etiqueta cambia a `PERSEGUIR` y **acelera**. Corré lejos (más de 350): vuelve a `ACECHAR` y afloja. Los slimes verdes siguen igual que siempre, sin etiqueta.

✅ **Punto de control 1:** el jefe muestra su estado encima, te acecha despacio, te persigue rápido al acercarte y se calma al alejarte. **Por ahora no te pega** (eso es la Parte 2). Los básicos no cambiaron.

Errores comunes en esta parte

▼ Abrí para ver soluciones

- **El jefe se queda quieto con la etiqueta vacía** → `jugador` es `null`. Si lo instanciaste a mano, ponelo **dentro de** `Enemigos` (que está debajo de `Jugador` en el árbol): así el jugador ya se anotó en el grupo cuando el jefe arranca.
- **El jefe sigue desapareciendo al tocarlo** → la función tiene que llamarse **exactamente** `_on_body_entered` con el mismo parámetro `(body: Node)` que en `enemigo.gd`; si no, no está *sobreescribiendo* nada.
- **"Invalid get index 'PERSEGUIR'"** → el `enum` se escribe **una sola vez**, arriba, y los nombres van **en mayúsculas exactas**.
- **La etiqueta no se ve** → es texto blanco; sobre fondo claro se pierde. Agregá `etiqueta.modulate = Color.BLACK` después de crearla.
- **Los slimes verdes también cambiaron** → tocaste `enemigo.gd`. En este TP ese archivo **no se modifica**.

Parte 2 — Tercer estado: ATACAR

Concepto: al alcanzarte, el jefe **se frena** y te pega **cada segundo** mientras estés cerca. Es un estado más y una función más — nada del resto se toca.

Cuatro cambios en `enemigo_elite.gd`:

1. Agregá el estado al `enum`:

```
enum Estado { ACECHAR, PERSEGUIR, ATACAR }
```

2. En `_ready()`, **debajo del bloque de la etiqueta**, creá el temporizador de ataque:

```
var timer_ataque := Timer.new()
timer_ataque.name = "TimerAtaque"
timer_ataque.wait_time = 1.0      # un golpe por segundo
timer_ataque.one_shot = true     # se dispara una vez y se frena
add_child(timer_ataque)
```

3. Reemplazá la función `_process` entera por esta (es la de la clase, con las tres flechas):

```

func _process(delta: float) -> void:
    if jugador == null:
        return
    var d := distancia_al_jugador()
    match estado:
        Estado.ACECHAR:
            acechar(delta)
            if d < 250: estado = Estado.PERSEGUIR
        Estado.PERSEGUIR:
            perseguir(delta)
            if d < 40: estado = Estado.ATACAR
            elif d > 350: estado = Estado.ACECHAR
        Estado.ATACAR:
            atacar()
            if d > 60: estado = Estado.PERSEGUIR
    $LabelEstado.text = Estado.keys()[estado]

```

4. Y agregó la función del nuevo estado, junto a las otras:

```

func atacar() -> void:                # quieto: no se mueve
    if $TimerAtaque.is_stopped():      # ¿ya pasó el segundo desde el último golpe?
        jugador.recibir_dano(dano)
        $TimerAtaque.start()

```

🗨️ **Cómo funciona el golpe cada segundo.** El `TimerAtaque` es *one shot*: al entrar en `ATACAR` está **frenado**, así que pega **enseguida** y lo arranca. Mientras corre (1 s), `is_stopped()` es falso y no pega. Al terminar se frena solo, y en el próximo frame vuelve a pegar. Sin variables extra, sin contar `delta` a mano.

Y fijate que `atacar()` **no mueve** al jefe: estar quieto también es un comportamiento.

Apretá **F6**. Dejá que el jefe te alcance: la etiqueta pasa a `ATACAR`, se frena pegado a vos, y tu barra baja **25 cada segundo** (cuatro golpes y perdés: es el jefe). Date un paso atrás: vuelve a `PERSEGUIR`.

💡 Para testear tranquilo, bajá `dano = 25` a `dano = 5` un rato, y después volvelo.

✅ **Punto de control 2:** el jefe te persigue, al alcanzarte se frena y te pega una vez por segundo, y si te alejás retoma la persecución. Compará con el diagrama: **cada `if` es una flecha**.

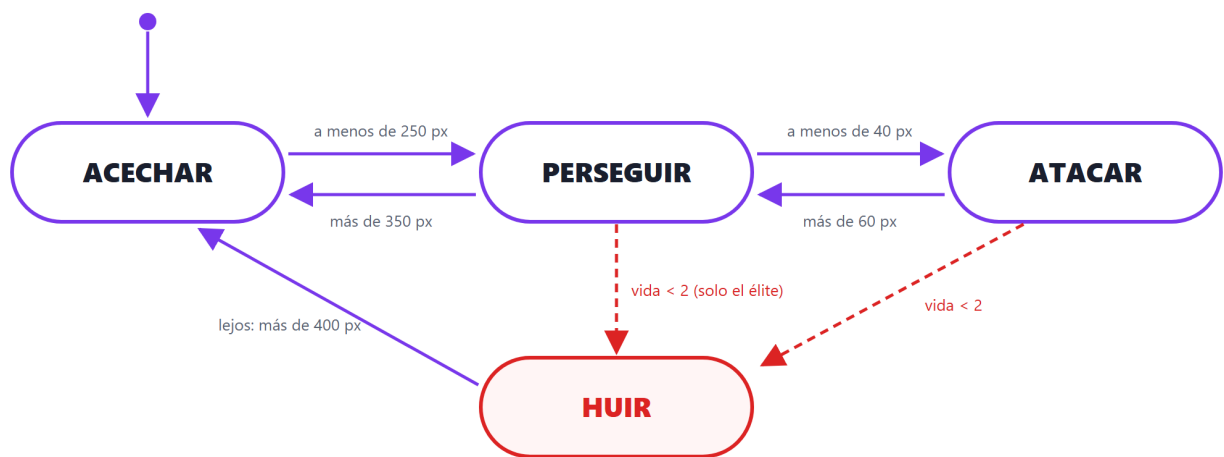
Errores comunes en esta parte

▼ Abrió para ver soluciones

- **"Node not found: TimerAtaque"** → el bloque que lo crea tiene que estar **dentro de** `_ready()`, con la misma sangría que el de la etiqueta.
- **Pega todo el tiempo, no cada segundo** → te faltó `one_shot = true`, o el `$TimerAtaque.start()` después de pegar.
- **Parpadea entre ATACAR y PERSEGUIR** → revisá los umbrales: entra a **40** y sale a **60**. Si pusiste el mismo número en los dos, es la histéresis de la Parte 0.
- **Nunca llega a ATACAR** → 40 px es poco si el `CollisionShape2D` del jefe es grande. Probá con `d < 60` y `d > 80`.

Parte 3 — Cuarto estado: HUIR

Concepto: la prueba de fuego de la clase: agregar un estado **sin romper nada**. Con un detalle nuevo: esta transición **no se dispara por distancia**, sino por un **evento** — recibir daño.



Las flechas rojas punteadas salen de `recibir_dano()`: una transición disparada por un evento (recibir daño), no por distancia

Cuatro cambios en `enemigo_elite.gd`:

1. El `enum`, completo:

```
enum Estado { ACECHAR, PERSEGUIR, ATACAR, HUIR }
```

2. En `_process`, agregá la rama de `HUIR` al final del `match`, debajo de la de `ATACAR`:

```
Estado.HUIR:
    huir(delta)
    if d > 400: estado = Estado.ACECHAR
```

3. La función, junto a las otras (es **perseguir al revés**):

```
func huir(delta: float) -> void:
    var dir := (position - jugador.position).normalized() # del jugador hacia mí
    position += dir * velocidad * 1.5 * delta              # más rápido que caminando
    $AnimatedSprite2D.flip_h = dir.x < 0
```

4. Y la **flecha** que entra a `HUIR`, en `recibir_dano()`:

```
func recibir_dano(cantidad: int) -> void:
    super(cantidad)
    $BarraVida.value = vida
    if vida > 0 and vida < 2:          # NUEVO: con 1 de vida, escapa
        estado = Estado.HUIR
```

🗨 **Esta flecha es distinta.** Las otras transiciones viven en el `match` y se disparan por **distancia**. Esta vive en `recibir_dano()` y se dispara por un **evento**: recibir un golpe. Las dos formas conviven sin problema — una máquina de estados no exige que todas las flechas salgan del mismo lugar. Y `vida > 0` evita mandar a huir a un jefe que ya murió con ese golpe.

Apretá **F6**. Dejá que las balas le peguen al jefe **cuatro veces**: con 1 de vida, la etiqueta cambia a `HUIR` y sale disparado en dirección contraria. Si se aleja más de 400 px, vuelve a `ACECHAR` ... y si te acercás de nuevo, te persigue **con 1 de vida**.

✅ **Punto de control 3:** el jefe huye al quedar con 1 de vida y vuelve a acechar cuando está lejos. Agregaste un estado y **nada de lo anterior se rompió**.

🔧 Errores comunes en esta parte

▼ Abrió para ver soluciones

- **"Identifier 'HUIR' not declared"** → falta agregarlo al `enum`.
- **El jefe no huye** → la condición va **después** de `super(cantidad)`, si no `vida` todavía no bajó. Y revisá que sea `vida > 0 and vida < 2`.
- **Huye pero vuelve enseguida** → 400 px es mucho en una arena chica; bajalo a 300. O subí el `1.5` a `2.0` para que escape más rápido.
- **Huye y se va de la pantalla** → es lo esperado con esta versión simple. Si querés que no salga, en `huir()` agregá un `clamp` de `position` como el del jugador (TP7, Parte 1).

🎮 Parte 4 — Proyecto final: ajustar, exportar, entregar

Ya está todo. Ahora convertilo en **tu** juego.

4.1 · Ajustar los números

Todo lo que define cómo se siente el jefe son **cinco números**. Jugá y totalos hasta que te guste:

DÓNDE	VARIABLE	QUÉ CAMBIA
<code>enemigo_elite.gd</code>	<code>velocidad</code>	Qué tan rápido persigue (acecha a la mitad, huye a 1.5x)
<code>enemigo_elite.gd</code>	umbrales <code>250</code> / <code>350</code>	Desde cuán lejos te ve y cuándo se calma
<code>enemigo_elite.gd</code>	<code>wait_time</code> del <code>TimerAtaque</code>	Cada cuánto pega
<code>spawner.gd</code>	<code>wait_time</code> del <code>Timer</code>	Cuántos slimes por segundo
<code>spawner.gd</code>	<code>contador >= 8</code>	Cada cuántos aparece un jefe

(Opcional) Para la versión final, ocultá la etiqueta agregando `etiqueta.visible = false` justo después de crearla en `_ready()`. O dejala: es simpática y muestra que el jefe piensa.

4.2 · Exportar

Como vimos en la clase 8:

1. **Editor** → **Manage Export Templates** → **Download and Install** (solo la primera vez).
2. **Project** → **Project Settings** → **Application** → **Config**: poné el **nombre** de tu juego.

3. **Project** → **Export** → **Add...** → **Windows Desktop** → carpeta y nombre (`mi_juego.exe`) → **Export Project**.

4. Godot genera `mi_juego.exe` y `mi_juego.pck` . **Van siempre juntos**: sin el `.pck` el `.exe` no arranca.

✅ **Punto de control 4 (final)**: el `.exe` corre en una compu **sin Godot instalado**, y se juega igual que en el editor.

Entrega — Proyecto final

Entregá **las dos cosas**:

1. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), y
2. El juego **exportado**: `mi_juego.exe` + `mi_juego.pck` en un `.zip` aparte.

(Opcional) Un **video corto** donde se vea: la horda persiguiéndote, el jefe acechando de lejos, acelerando al acercarte, atacándote, y huyendo con 1 de vida.

Nombre: `tp8-final-ApellidoNombre.zip` y `tp8-final-ApellidoNombre-exe.zip`

✓ Checklist de autoevaluación

- ☐ `enemigo_elite.gd` tiene `enum Estado` con **cuatro** estados y **su propio** `_process` con `match` .
- ☐ Cada estado tiene **su función** (`acechar` , `perseguir` , `atacar` , `huir`).
- ☐ El jefe muestra su **estado** en una etiqueta encima (aunque después la ocultes).
- ☐ El jefe **acecha despacio** hacia vos, **persigue** a menos de 250 px y se **calma** a más de 350.
- ☐ Al alcanzarte **se frena** y pega **una vez por segundo** (ya no desaparece al tocarlo).
- ☐ **Huye** con 1 de vida (flecha desde `recibir_dano()`) y vuelve a acechar cuando está lejos.
- ☐ El slime básico quedó **igual que en el TP7**: kamikaze y sin etiqueta. `enemigo.gd` **no se tocó**.
- ☐ El juego está **exportado** y corre sin Godot.

Código completo de referencia

Por si te perdiste en algún paso: así tiene que quedar `enemigo_elite.gd` al final. `enemigo.gd` es el **mismo del TP7, sin cambios**.

▼ `enemigo_elite.gd` completo

```

extends "res://enemigo.gd"

enum Estado { ACECHAR, PERSEGUIR, ATACAR, HUIR }

var estado := Estado.ACECHAR
var jugador: Node2D = null

func _ready() -> void:
    super()
    vida = 5
    velocidad = 35.0
    dano = 25
    $BarraVida.max_value = vida
    $BarraVida.value = vida

    jugador = get_tree().get_first_node_in_group("jugador")

    var etiqueta := Label.new()
    etiqueta.name = "LabelEstado"
    etiqueta.position = Vector2(-36, -66)
    add_child(etiqueta)

    var timer_ataque := Timer.new()
    timer_ataque.name = "TimerAtaque"
    timer_ataque.wait_time = 1.0
    timer_ataque.one_shot = true
    add_child(timer_ataque)

func _process(delta: float) -> void:
    if jugador == null:
        return
    var d := distancia_al_jugador()
    match estado:
        Estado.ACECHAR:
            acechar(delta)
            if d < 250: estado = Estado.PERSEGUIR
        Estado.PERSEGUIR:
            perseguir(delta)
            if d < 40: estado = Estado.ATACAR
            elif d > 350: estado = Estado.ACECHAR
        Estado.ATACAR:
            atacar()
            if d > 60: estado = Estado.PERSEGUIR
        Estado.HUIR:

```

```

        huir(delta)
        if d > 400: estado = Estado.ACECHAR
        $LabelEstado.text = Estado.keys()[estado]

func _on_body_entered(body: Node) -> void:
    pass

func acechar(delta: float) -> void:
    var dir := (jugador.position - position).normalized()
    position += dir * velocidad * 0.5 * delta
    $AnimatedSprite2D.flip_h = dir.x < 0

func perseguir(delta: float) -> void:
    var dir := (jugador.position - position).normalized()
    position += dir * velocidad * delta
    $AnimatedSprite2D.flip_h = dir.x < 0

func atacar() -> void:
    if $TimerAtaque.is_stopped():
        jugador.recibir_dano(dano)
        $TimerAtaque.start()

func huir(delta: float) -> void:
    var dir := (position - jugador.position).normalized()
    position += dir * velocidad * 1.5 * delta
    $AnimatedSprite2D.flip_h = dir.x < 0

func distancia_al_jugador() -> float:
    return position.distance_to(jugador.position)

func recibir_dano(cantidad: int) -> void:
    super(cantidad)
    $BarraVida.value = vida
    if vida > 0 and vida < 2:
        estado = Estado.HUIR

```

Extra (opcional)

- **Que se note el cambio.** Un color por estado con `modulate` (blanco acechando, amarillo persiguiendo, rojo atacando), o un `Tween` de escala al entrar a `ATACAR` (Clase 7). Los buenos enemigos **avisan** en qué estado están.

- **Un quinto estado:** `EMBESTIR`. Desde `PERSEGUIR`, si el jugador está entre 100 y 150 px, que cargue en línea recta a 3× durante medio segundo (un `Timer`) y después vuelva a perseguir. Es el ataque clásico de un jefe.
- **Un jefe que dispara.** Que en `ATACAR`, en vez de acercarse, se **frene a distancia** e instancie una bala hacia vos (todo lo que hace falta ya lo tenés del TP7).
- **Dificultad progresiva.** Que la `velocidad` y el radio de visión del jefe suban un poco cada vez que aparece uno nuevo.
- **¿Y si la horda también pensara?** Podrías darle la máquina a `enemigo.gd`. Pero antes contestá: ¿cuál sería su estado tranquilo? Si la respuesta es "patrullar al azar", releé la decisión de diseño del principio.
- **Game Over de verdad.** Puntaje en un Autoload y pantalla final, como en el TP6.

Recursos

- `enum` y `match` en GDScript: [GDScript basics](#)
- Máquinas de estado en Godot, con más profundidad: [GDQuest — Finite State Machine](#)
- El nodo `Timer`: [Timer](#)
- Exportar el proyecto: [Exporting projects](#)

Diagramas: elaboración propia para la diplomatura. Sprites de **Brackeys** (CC0), heredados del TP7.

¡Felicitaciones! Terminaste la diplomatura con un juego exportado, con un jefe que piensa. 🎉